Diário de Bordo — Aula 11

ALTER TABLE, DROP TABLE, UPDATE, DELETE e integridade dos dados · Prof. Clóvis

Data	___/___/2026 — preenchido em 30/10/2026
Grupo	7
Projeto ERP	Petly Club — ERP para pet shop com loja física e loja virtual
Gerente	_____
Integrantes presentes	_____

> **Observação:** os comandos foram executados no PostgreSQL 16 numa **cópia** do banco do projeto (`petly_aula11`, criada a partir de `petly_club` com as duas massas de dados), para não alterar a base de testes. As mensagens de erro abaixo foram copiadas da saída do SGBD sem alteração.

1. Objetivos da aula

Aplicar comandos DDL e DML, compreender seus efeitos e preservar as regras de negócio e a integridade dos dados no ERP.

2. Retomada dos conhecimentos anteriores

2.1. Qual a diferença entre DDL e DML?

DDL (Data Definition Language) define e altera a **estrutura** do banco: `CREATE`, `ALTER` e `DROP` de tabelas, colunas e restrições. **DML** (Data Manipulation Language) trabalha com os **dados** dentro dessa estrutura: `INSERT`, `UPDATE`, `DELETE` e `SELECT`. No projeto, o DDL está em `petly_club_ddl_limpo.sql` e o DML nas massas de dados, nos testes e nas consultas.

2.2. Qual é a função de uma chave estrangeira?

Garantir a **integridade referencial**: o valor da FK só pode ser um registro que existe na tabela referenciada. Exemplo: `pedido.id_cliente` (fk_pedido_cliente) impede um pedido de cliente inexistente — regra RN02, todo pedido pertence a um cliente cadastrado.

2.3. O que pode ocorrer ao excluir um registro referenciado por outra tabela?

Depende da ação definida na FK. Com `RESTRICT` (usado em todas as FKs do projeto), o banco **recusa** a exclusão. Com `CASCADE`, os registros dependentes seriam apagados junto (ex.: excluir um cliente apagaria seus pedidos e o histórico de vendas). Com `SET NULL`, a FK ficaria vazia. Sem nenhuma FK, sobrariam registros “órfãos”.

3. Missão prática — Auditoria do ERP

Etapa 1 — ALTER TABLE

Tabela	fornecedor

Alteração	Nova coluna `prazo_entrega_dias` (SMALLINT, NOT NULL, DEFAULT 7, CHECK > 0)
Justificativa	A proposta de IA do projeto (previsão de demanda e sugestão de compra) precisa saber quantos dias cada fornecedor leva para entregar, para calcular quando comprar antes de o produto acabar. Hoje esse dado não existe no banco. O DEFAULT preenche os fornecedores já cadastrados, e o CHECK impede prazo zero ou negativo.

```
ALTER TABLE fornecedor
    ADD COLUMN prazo_entrega_dias SMALLINT NOT NULL DEFAULT 7
    CONSTRAINT ck_fornecedor_prazo CHECK (prazo_entrega_dias > 0);
```

Resultado observado: `ALTER TABLE` aceito; os 3 fornecedores passaram a ter `prazo_entrega_dias = 7`. Um teste com prazo 0 foi recusado:

```
ERROR:  new row for relation "fornecedor" violates check constraint "ck_fornecedor_prazo"
DETAIL:  Failing row contains (1, NutriPet Distribuidora de Alimentos LTDA, 12345678000199,
1133334444, contato@nutripet.com.br, 0).
```

Para entrar no projeto, a mudança precisaria ir também para o DDL, o modelo lógico e o dicionário de dados — por isso foi feita só na cópia.

Etapa 2 — UPDATE

Tabela / campo	produto / preco
Valor anterior → novo valor	R$ 29,90 → R$ 32,90 (Shampoo Neutro 500ml, id_produto = 2)

```
UPDATE produto SET preco = 32.90 WHERE id_produto = 2;
```

Resultado observado: `UPDATE 1`. O preço de catálogo mudou, mas os pedidos 2 e 4, vendidos antes, continuam com `preco_unitario = 29.90` em `item_pedido` — o preço praticado fica registrado na venda (RN13).

Qual o risco de executar UPDATE sem WHERE?

O comando altera **todas** as linhas da tabela: todos os produtos passariam a custar R$ 32,90. O banco aceitaria, porque nenhuma restrição é violada. Por isso, antes de um UPDATE, o grupo roda um SELECT com o mesmo WHERE para conferir quais linhas serão afetadas, e usa `BEGIN`/`ROLLBACK` nos testes.

Etapa 3 — DELETE

Tabela / registro	cliente — primeiro a cliente 1 (Maria Souza, com pedidos); depois um cliente de teste sem pedidos

```
DELETE FROM cliente WHERE id_cliente = 1;          -- cliente com pedidos

INSERT INTO cliente (nome, cpf, telefone, email)
VALUES ('Cliente Teste Aula 11', '00000000191', '11900000000', 'teste.aula11@email.com');
DELETE FROM cliente WHERE cpf = '00000000191';     -- cliente sem pedidos
```

```
ERROR:  update or delete on table "cliente" violates foreign key constraint "fk_pedido_cliente"
on table "pedido"
DETAIL:  Key (id_cliente)=(1) is still referenced from table "pedido".
...
DELETE 1
```

O que ocorre se houver uma FK que impeça a exclusão?

O SGBD recusa o DELETE inteiro e informa qual FK e qual tabela ainda referenciam o registro (acima: `fk_pedido_cliente`, tabela `pedido`). Nada é apagado. No ERP, isso protege o histórico de vendas: um cliente com pedidos não deve ser excluído; se ele pedir para sair do clube, o caminho seria marcá-lo como inativo, e não apagar o cadastro. Já o cliente de teste, sem pedidos, foi excluído normalmente.

Etapa 4 — DROP TABLE

```
CREATE TABLE teste_descartavel (id INTEGER PRIMARY KEY, observacao VARCHAR(50));
INSERT INTO teste_descartavel VALUES (1, 'tabela criada só para o teste');
DROP TABLE teste_descartavel;
```

Resultado observado: `CREATE TABLE`, `INSERT 0 1`, `DROP TABLE`; depois, `to_regclass('teste_descartavel')` retornou vazio — a tabela não existe mais.

Qual a diferença entre DELETE e DROP TABLE?

DELETE é DML: remove **linhas** (todas ou as do WHERE) e a tabela continua existindo, com estrutura, restrições e permissões. `DROP TABLE` é DDL: remove a **tabela inteira** — estrutura, dados, índices e restrições. Num ERP, DROP só se usa em tabela de teste ou numa mudança de modelo planejada; e, se outra tabela tiver FK para ela, o DROP é recusado (a menos que se use CASCADE).

4. Investigação — Integridade dos dados

Tipo investigado: ☐ Entidade (PK) ☑ Referencial (FK) ☐ Domínio ☐ Unicidade

Erro provocado: registrar a venda de 1 unidade da **ração** (produto 1) no pedido 1 tirando do lote 3, que é um lote de **shampoo** (produto 2).

```
INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario,
id_pedido)
VALUES ('venda', 1, 3, 1, 2, 1);
```

```
ERROR:  insert or update on table "movimentacao_estoque" violates foreign key constraint
"fk_mov_lote"
DETAIL:  Key (id_lote, id_produto)=(3, 1) is not present in table "lote".
```

Por que o banco recusou: a FK composta `(id_lote, id_produto) → lote` (ajuste [A1] do modelo lógico) só aceita a combinação lote + produto que existe. O lote 3 é do produto 2, então a venda da ração não pode sair dele (RN20).

Correção: usar um lote do próprio produto 1, escolhido por FEFO — o lote 1 (LOTE-RAC-2026-A, vence primeiro) ou o lote 2 (LOTE-RAC-2026-B).

5. Reflexão crítica do grupo

5.1. Qual foi a maior dificuldade?

Escolher uma alteração estrutural que fizesse sentido para o negócio e não só “cumprir o ALTER”. Também foi preciso cuidado para testar UPDATE e DELETE sem estragar a base de testes do grupo — resolvido trabalhando numa cópia do banco.

5.2. Como esses comandos ajudam na manutenção de um ERP real?

O ERP muda junto com a empresa: um novo dado (como o prazo do fornecedor) entra com `ALTER TABLE`, sem recriar o banco; preços e status são atualizados com `UPDATE`; cadastros feitos por engano saem com `DELETE`. As restrições (CHECK, FK) continuam valendo nessas operações, então a manutenção não quebra as regras de negócio.

5.3. Quais riscos existem ao alterar ou excluir dados sem considerar os relacionamentos?

Perder histórico (CASCADE apagando pedidos e pagamentos de um cliente), deixar registros órfãos, alterar valores de vendas passadas, ou um UPDATE sem WHERE mudar a tabela inteira. No projeto, as FKs com RESTRICT, o preço gravado no item do pedido e a movimentação de estoque que não pode ser alterada (RN19) existem justamente para evitar isso.

5.4. O que aprendemos que poderá melhorar a entrega final?

Que toda mudança de estrutura precisa ir para todos os artefatos ao mesmo tempo (DDL, modelo lógico, dicionário e DER), senão eles ficam diferentes entre si. Na entrega final, conferimos essa coerência e atualizamos o DER com os ajustes [A2] e [A3].

6. Checklist de aprendizagem

☑ Revisamos DDL e DML.
☑ Executamos ALTER TABLE e justificamos a mudança.
☑ Executamos UPDATE com WHERE.
☑ Executamos DELETE de forma controlada.
☑ Testamos DROP TABLE somente em tabela de teste.
☑ Investigamos uma restrição de integridade.
☑ Registramos comandos e resultados.
☐ Todos participaram da atividade.

7. Autoavaliação

☐ Inicial ☐ Básico ☐ Satisfatório ☐ Avançado

Principal aprendizagem:

As restrições do banco não servem só na carga dos dados: elas também protegem as alterações e exclusões do dia a dia, e a mensagem de erro do SGBD diz exatamente qual regra foi violada.

O que precisamos revisar?

As ações das FKs (RESTRICT, CASCADE, SET NULL) e quando cada uma é adequada; e como fazer ALTER TABLE quando já existem dados que violariam a nova restrição.

8. Validação

Responsável pelo registro: _____ Gerente: _____

Validação do professor: _____